
TRÍ TUỆ NHÂN TẠO

Linear Regression (Taxi Fare)

Bài tập Colab — Dự đoán giá cước taxi (dịch tiếng Việt)

Biên soạn:

ĐẶNG TẤN QUÍ

SDT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Phần 1 — Chuẩn bị	4
1.1 Nạp các module cần thiết	4
1.2 Nạp dataset	4
1.3 Chọn cột cần dùng	5
2 Phần 2 — Khám phá dataset	6
2.1 Xem thống kê mô tả	6
2.2 Ma trận tương quan (correlation matrix)	7
2.3 Trực quan hóa quan hệ bằng pair plot	7
3 Phần 3 — Huấn luyện mô hình	8
3.1 Định nghĩa các hàm tạo và huấn luyện mô hình	8
3.2 Huấn luyện mô hình với một feature	9
3.3 Thử nghiệm siêu tham số	10
3.4 Huấn luyện mô hình với hai feature	11
3.5 So sánh các experiment	12
4 Phần 4 — Kiểm tra mô hình (validation)	13
4.1 Dùng mô hình để dự đoán	13

Lời nói đầu

Nguồn

Tài liệu dịch từ notebook Colab *Linear Regression* (ML Crash Course, 2023): https://colab.research.google.com/github/google/eng-edu/blob/main/ml/cc/exercises/linear_regression_taxi.ipynb.

Mục tiêu bài tập

Bạn sẽ dùng một dataset thật để huấn luyện mô hình dự đoán **giá cước** của một chuyến taxi ở Chicago (Illinois, Hoa Kỳ).

Mục tiêu học tập

Sau khi làm xong, bạn có thể:

- Đọc file `.csv` vào pandas DataFrame.
- Khám phá dataset bằng các thư viện trực quan hóa.
- Thử nhiều lựa chọn đặc trưng (features) để xây mô hình hồi quy tuyến tính.
- Tinh chỉnh siêu tham số (hyperparameters) của mô hình.
- So sánh các lần chạy qua RMSE và loss curve.

Mô tả dataset

Dataset cho bài tập: https://download.mlcc.google.com/mledu-datasets/chicago_taxi_train.csv là một phần con của bộ dữ liệu *City of Chicago Taxi Trips*: <https://data.cityofchicago.org/Transportation/Taxi-Trips/wrvz-psew>. Dữ liệu tập trung vào một giai đoạn 2 ngày trong tháng 5/2022.

1 Phần 1 — Chuẩn bị

1.1 Nạp các module cần thiết

Vì sao cần nhiều thư viện?

Bài tập sử dụng nhiều thư viện Python để thao tác dữ liệu, huấn luyện mô hình, và trực quan hóa.

Hướng dẫn

1. Chạy cell **Install required libraries**.
2. Chạy cell **Load dependencies**.

```
#@title Install required libraries

!pip install google-ml-edu==0.1.3 \
  keras~=3.8.0 \
  matplotlib~=3.10.0 \
  numpy~=2.0.0 \
  pandas~=2.2.0 \
  tensorflow~=2.18.0

print('\n\nAll requirements successfully installed.')
```

```
#@title Code - Load dependencies

# data
import numpy as np
import pandas as pd

# machine learning
import keras
import ml_edu.experiment
import ml_edu.results

# data visualization
import plotly.express as px
```

1.2 Nạp dataset

Đọc CSV vào DataFrame

Cell sau sẽ nạp dataset và tạo một pandas DataFrame. Bạn có thể coi DataFrame giống như một bảng tính (spreadsheet) có hàng và cột: hàng là các mẫu dữ liệu, cột là các thuộc tính của mỗi mẫu.

```
# @title
```

```
chicago_taxi_dataset = pd.read_csv("https://download.mlcc.google.com/mledu-  
datasets/chicago_taxi_train.csv")
```

1.3 Chọn cột cần dùng

Giữ lại một số cột

Cell sau chỉ giữ lại một số cột cần thiết trong dataset.

```
#@title Code - Read dataset  
  
# Updates dataframe to use specific columns.  
training_df = chicago_taxi_dataset.loc[:, ('TRIP_MILES', 'TRIP_SECONDS', 'FARE', '  
COMPANY', 'PAYMENT_TYPE', 'TIP_RATE')]  
  
print('Read dataset completed successfully.')  
print('Total number of rows: {0}\n\n'.format(len(training_df.index)))  
training_df.head(200)
```

2 Phần 2 — Khám phá dataset

2.1 Xem thông kê mô tả

Vì sao phải “hiểu dữ liệu” trước?

Trong hầu hết dự án ML, một phần rất lớn là hiểu dữ liệu: phạm vi giá trị, outlier, phân phối, biến phân loại, thiếu dữ liệu, ...

Hướng dẫn

1. Chạy cell **View dataset statistics**.
2. Nhìn bảng và trả lời:
 - Giá cước tối đa là bao nhiêu?
 - Quãng đường trung bình cho mọi chuyến là bao nhiêu?
 - Có bao nhiêu hãng taxi trong dataset?
 - Loại thanh toán phổ biến nhất là gì?
 - Có cột nào bị thiếu dữ liệu không?
3. Chạy cell **View answers...** để kiểm tra.

Vì sao có NaN trong bảng describe?

Khi làm dữ liệu trong Python, bạn có thể thấy NaN (not a number) nếu một phép tính không thể thực hiện hoặc dữ liệu bị thiếu. Trong dataset taxi, **PAYMENT_TYPE** và **COMPANY** là biến phân loại (không phải số), nên các thống kê như mean/max không có nghĩa và được hiển thị là NaN.

```
#@title Code - View dataset statistics
```

```
print('Total number of rows: {0}\n\n'.format(len(training_df.index)))
training_df.describe(include='all')
```

Đáp án tham khảo

- Giá cước tối đa? **159,25 USD**.
- Quãng đường trung bình mọi chuyến? **8,2895 dặm**.
- Có bao nhiêu hãng taxi trong dataset? **31**.
- Loại thanh toán phổ biến nhất? **Thẻ tín dụng** (Credit Card).
- Có feature nào thiếu dữ liệu không? **Không**.

2.2 Ma trận tương quan (correlation matrix)

Chọn feature nên dựa vào tương quan

Một bước quan trọng là xem feature nào tương quan tốt với **nhân** (FARE). Thông thường giá cước liên quan đến quãng đường và thời lượng chuyển đi. Ma trận tương quan giúp bạn định lượng điều này.

Ý nghĩa giá trị tương quan

- 1.0: tương quan dương hoàn hảo (một tăng thì cái kia tăng).
- -1.0: tương quan âm hoàn hảo (một tăng thì cái kia giảm).
- 0.0: không tương quan tuyến tính đáng kể.

Nhìn chung, $|\text{corr}|$ càng lớn thì sức dự đoán càng tốt.

Hướng dẫn kiểm tra hiểu

Chạy cell sau và trả lời:

- Feature nào tương quan mạnh nhất với FARE?
- Feature nào tương quan yếu nhất với FARE?

```
#@title Code - View correlation matrix
training_df.corr(numeric_only = True)
```

Đáp án tham khảo

Tương quan mạnh nhất với FARE: TRIP_MILES (quãng đường). Đúng như kỳ vọng, đây là feature tốt để bắt đầu huấn luyện. TRIP_SECONDS (thời gian chuyển) cũng tương quan khá mạnh với giá cước.

Tương quan yếu nhất với FARE: TIP_RATE (tỷ lệ tiền boa).

2.3 Trực quan hóa quan hệ bằng pair plot

Pair plot / scatter matrix

Chạy cell sau để xem scatter matrix giữa các cột quan trọng.

```
#@title Code - View pairplot
px.scatter_matrix(training_df, dimensions=["FARE", "TRIP_MILES", "TRIP_SECONDS"])
```

3 Phần 3 — Huấn luyện mô hình

3.1 Định nghĩa các hàm tạo và huấn luyện mô hình

Hướng dẫn

Chạy cell Define ML functions.

```
#@title Code - Define ML functions

def create_model(
    settings: ml_edu.experiment.ExperimentSettings,
    metrics: list[keras.metrics.Metric],
) -> keras.Model:
    """Create and compile a simple linear regression model."""
    inputs = {name: keras.Input(shape=(1,), name=name) for name in settings.
        input_features}
    concatenated_inputs = keras.layers.Concatenate()(list(inputs.values()))
    outputs = keras.layers.Dense(units=1)(concatenated_inputs)
    model = keras.Model(inputs=inputs, outputs=outputs)

    model.compile(optimizer=keras.optimizers.RMSprop(learning_rate=settings.
        learning_rate),
        loss="mean_squared_error",
        metrics=metrics)

    return model

def train_model(
    experiment_name: str,
    model: keras.Model,
    dataset: pd.DataFrame,
    label_name: str,
    settings: ml_edu.experiment.ExperimentSettings,
) -> ml_edu.experiment.Experiment:
    """Train the model by feeding it data."""

    features = {name: dataset[name].values for name in settings.input_features}
    label = dataset[label_name].values
    history = model.fit(x=features,
        y=label,
        batch_size=settings.batch_size,
        epochs=settings.number_epochs)

    return ml_edu.experiment.Experiment(
        name=experiment_name,
        settings=settings,
        model=model,
        epochs=history.epoch,
        metrics_history=pd.DataFrame(history.history),
    )
```



```
print("SUCCESS: defining linear regression functions complete.")
```

3.2 Huấn luyện mô hình với một feature

Bắt đầu từ feature tương quan mạnh nhất

Ta bắt đầu với TRIP_MILES vì nó tương quan mạnh nhất với nhân FARE.

Hướng dẫn

1. Chạy cell **Experiment 1**.
2. Quan sát đường loss/RMSE và đồ thị dự đoán.
3. Trả lời:
 - Mất bao nhiêu epoch để mô hình hội tụ?
 - Mô hình khớp dữ liệu mẫu tốt đến đâu?

RMSE có đơn vị giống nhân

Trong huấn luyện, bạn sẽ thấy RMSE. Đơn vị RMSE giống đơn vị nhân (\$). RMSE giúp ước lượng dự đoán sai lệch trung bình bao nhiêu đô la so với giá quan sát.

```
#@title Code - Experiment 1

settings_1 = ml_edu.experiment.ExperimentSettings(
    learning_rate = 0.001,
    number_epochs = 20,
    batch_size = 50,
    input_features = ['TRIP_MILES']
)

metrics = [keras.metrics.RootMeanSquaredError(name='rmse')]

model_1 = create_model(settings_1, metrics)

experiment_1 = train_model('one_feature', model_1, training_df, 'FARE', settings_1
)

ml_edu.results.plot_experiment_metrics(experiment_1, ['rmse'])
ml_edu.results.plot_model_predictions(experiment_1, training_df, 'FARE')
```

Đáp án tham khảo

Bao nhiêu epoch để hội tụ? Dùng loss curve để xem chỗ loss bắt đầu “đi ngang”. Với `learning_rate = 0.001`, `epochs = 20`, `batch_size = 50`, khoảng **5 epoch** là đủ để mô hình hội tụ.

Mô hình khớp dữ liệu mẫu thế nào? Nhìn đồ thị dự đoán, mô hình khớp dữ liệu mẫu khá tốt.

3.3 Thử nghiệm siêu tham số

Vì sao phải chạy nhiều experiment?

Trong ML, thường phải thử nhiều cấu hình siêu tham số để tìm bộ tốt nhất. Hãy thay đổi từng siêu tham số và quan sát loss curve và đồ thị mô hình.

Các thử nghiệm gợi ý

- **Experiment 1:** tăng learning rate lên 1.0 (giữ batch size 50).
- **Experiment 2:** giảm learning rate xuống 0.0001 (giữ batch size 50).
- **Experiment 3:** tăng batch size lên 500 (giữ learning rate 0.001).

Hướng dẫn

Sửa các siêu tham số trong cell **Experiment 2**, chạy, rồi quan sát khác biệt. Lặp lại cho từng thử nghiệm. Trả lời:

- Tăng learning rate ảnh hưởng thế nào?
- Giảm learning rate ảnh hưởng thế nào?
- Thay batch size ảnh hưởng kết quả không?

```
#@title Code - Experiment 2

# The following variables are the hyperparameters.
# TODO - Adjust these hyperparameters to see how they impact a training run.
settings_2 = ml_edu.experiment.ExperimentSettings(
    learning_rate = 0.001,
    number_epochs = 20,
    batch_size = 50,
    input_features = ['TRIP_MILES']
)

metrics = [keras.metrics.RootMeanSquaredError(name='rmse')]

model_2 = create_model(settings_2, metrics)

experiment_2 = train_model('one_feature_hyper', model_2, training_df, 'FARE',
    settings_2)

ml_edu.results.plot_experiment_metrics(experiment_2, ['rmse'])
ml_edu.results.plot_model_predictions(experiment_2, training_df, 'FARE')
```

Đáp án tham khảo

Tăng learning rate: Loss curve dao động mạnh, không hướng về hội tụ sau mỗi vòng lặp; đồ thị dự đoán khớp dữ liệu kém. Learning rate quá cao thường **khó** huấn luyện được mô hình tốt.

Giảm learning rate: Loss giảm chậm, có thể cần nhiều epoch hơn mới hội tụ; có thể tăng số epoch để mô hình cuối cùng vẫn hội tụ, nhưng **mất thời gian hơn**.

Tăng batch size: Mỗi epoch chạy nhanh hơn (vì tăng batch size thì giảm số lần lặp trong mỗi epoch), nhưng với 20 epoch mô hình có thể chưa hội tụ (tương tự learning rate nhỏ). Nếu có thời gian, hãy tăng số epoch — cuối cùng bạn sẽ thấy mô hình hội tụ.

3.4 Huấn luyện mô hình với hai feature**Thêm feature có giúp tốt hơn không?**

Mô hình dùng TRIP_MILES có sức dự đoán khá tốt, nhưng ta thử thêm feature thứ hai để cải thiện. Dataset không có TRIP_MINUTES sẵn, nhưng có thể suy ra từ TRIP_SECONDS.

Hướng dẫn

1. Xem code trong cell **Experiment 3**.
2. Chạy cell và so sánh RMSE với mô hình một feature.
3. Trả lời:
 - Mô hình 2 feature có tốt hơn không?
 - Dùng TRIP_SECONDS thay TRIP_MINUTES có khác không?
 - Mô hình có gần công thức tính giá thật không?

Vì sao có đồ thị 3D?

Scatter plot giữa (2 feature, 1 nhãn) là đồ thị 3D: hai feature trên trục x, y , nhãn trên trục z . Điểm dữ liệu là các vòng tròn, còn mô hình là một mặt phẳng. Nếu mô hình tốt, đa số điểm sẽ nằm gần mặt phẳng đó.

```
#@title Code - Experiment 3

settings_3 = ml_edu.experiment.ExperimentSettings(
    learning_rate = 0.001,
    number_epochs = 20,
    batch_size = 50,
    input_features = ['TRIP_MILES', 'TRIP_MINUTES']
)

training_df['TRIP_MINUTES'] = training_df['TRIP_SECONDS']/60

metrics = [keras.metrics.RootMeanSquaredError(name='rmse')]
```

```

model_3 = create_model(settings_3, metrics)

experiment_3 = train_model('two_features', model_3, training_df, 'FARE',
                           settings_3)

ml_edu.results.plot_experiment_metrics(experiment_3, ['rmse'])
ml_edu.results.plot_model_predictions(experiment_3, training_df, 'FARE')

```

Đáp án tham khảo

Hai feature có tốt hơn một feature không? So sánh RMSE giữa các lần chạy. Ví dụ: RMSE mô hình một feature là 3,7457, hai feature là 3,4787 $\Rightarrow$ trung bình dự đoán gần giá quan sát hơn khoảng **0,27 USD**.

Dùng TRIP_SECONDS thay TRIP_MINUTES? Khi có nhiều feature số, các giá trị nên **cùng thứ tự độ lớn**. TRIP_MILES có trung bình $\approx 8,3$, TRIP_SECONDS $\approx 1\,320$ (chênh hai bậc độ lớn). TRIP_MINUTES có trung bình ≈ 22 , gần thang của TRIP_MILES hơn. Đây không phải cách duy nhất để chuẩn hóa; module khác sẽ nói thêm.

So với công thức tính giá thật ở Chicago: Hành khách trả tiền mặt, công thức tham khảo:

$$\text{FARE} = 2,25 \times \text{TRIP_MILES} + 0,12 \times \text{TRIP_MINUTES} + 3,25.$$

Thường ta không biết công thức đúng, nhưng ở đây có thể so trọng số/bias học được với công thức trên — mô hình thường **khá sát**.

3.5 So sánh các experiment

```

ml_edu.results.compare_experiment([experiment_1, experiment_3], ['rmse'],
                                  training_df, training_df['FARE'].values)

```

4 Phần 4 — Kiểm tra mô hình (validation)

4.1 Dùng mô hình để dự đoán

Hướng dẫn

1. Chạy cell **Define functions to make predictions**.
2. Chạy cell **Make predictions**.
3. Xem bảng dự đoán và trả lời: dự đoán gần nhãn thật đến mức nào?

```
#@title Code - Define functions to make predictions
def format_currency(x):
    return "${:.2f}".format(x)

def build_batch(df, batch_size):
    batch = df.sample(n=batch_size).copy()
    batch.set_index(np.arange(batch_size), inplace=True)
    return batch

def predict_fare(model, df, features, label, batch_size=50):
    batch = build_batch(df, batch_size)
    predicted_values = model.predict_on_batch(x={name: batch[name].values for name
        in features})

    data = {"PREDICTED_FARE": [], "OBSERVED_FARE": [], "L1_LOSS": [],
        features[0]: [], features[1]: []}
    for i in range(batch_size):
        predicted = predicted_values[i][0]
        observed = batch.at[i, label]
        data["PREDICTED_FARE"].append(format_currency(predicted))
        data["OBSERVED_FARE"].append(format_currency(observed))
        data["L1_LOSS"].append(format_currency(abs(observed - predicted)))
        data[features[0]].append(batch.at[i, features[0]])
        data[features[1]].append("${:.2f}".format(batch.at[i, features[1]]))

    output_df = pd.DataFrame(data)
    return output_df

def show_predictions(output):
    header = "-" * 80
    banner = header + "\n" + "|" + "PREDICTIONS".center(78) + "|" + "\n" + header
    print(banner)
    print(output)
    return
```

```
#@title Code - Make predictions

output = predict_fare(experiment_3.model, training_df, experiment_3.settings.
    input_features, 'FARE')
show_predictions(output)
```

Đáp án tham khảo

Trên mẫu ngẫu nhiên, mô hình dự đoán giá cước **khá tốt**: hầu hết giá dự đoán không lệch nhiều so với giá quan sát. Xem cột $L1_LOSS = |\text{quan sát} - \text{dự đoán}|$ để thấy rõ.

Đặng Tấn Quý